

# Centralizing Model Context Protocol Infrastructure: Client Configuration, Gateway Multiplexing, and Enterprise Deployment Strategies

## The Proliferation of Agentic AI and the Configuration Drift Dilemma

The Model Context Protocol (MCP) has fundamentally altered the trajectory of artificial intelligence architectures. Transitioning from an experimental standard to the definitive backbone of production-grade agentic AI by May 2026, MCP establishes a standardized communication layer that enables large language models (LLMs) to dynamically discover and execute external tools at runtime.<sup>1</sup> This paradigm shift liberates LLMs from the constraints of static text generation, transforming them into active participants within enterprise ecosystems. Through MCP, intelligent agents can autonomously query live SQL databases, read local filesystems, execute web searches, interact with Customer Relationship Management (CRM) platforms, and orchestrate complex microservice workflows.<sup>1</sup>

However, the rapid and widespread adoption of MCP has catalyzed a severe operational bottleneck at the client level. In a standard multi-agent development environment utilizing tools such as the Gemini CLI, OpenCode CLI, and GitHub Copilot, developers confront a localized configuration crisis.<sup>3</sup> The foundational architecture of these clients relies on decentralized, local configuration files. Consequently, without a centralized gateway, every individual AI client requires its own discrete, manually maintained MCP server configuration.<sup>3</sup> This decentralized architecture necessitates duplicated connection strings, scattered API credentials across multiple plaintext files, and the manual administration of localized transport mechanisms, including Standard Input/Output (Stdio), legacy Server-Sent Events (SSE), and Streamable HTTP.<sup>3</sup>

The resulting configuration drift is a profound liability for engineering teams. As developers configure identical toolsets across disparate clients, the probability of credential mismanagement and version desynchronization increases exponentially.<sup>3</sup> Furthermore, this architecture expands the enterprise attack surface, running contrary to modern data residency and risk management frameworks. Regulatory pressures, such as the EU AI Act, which features staged applicability phasing in across 2026 and penalty provisions reaching up to 7% of global annual turnover for severe violations, mandate strict governance over how AI systems interact with external data.<sup>4</sup>

This report provides an exhaustive, granular analysis of localized MCP client configurations,

demonstrating the integration of the Firecrawl MCP server across primary development environments to highlight existing friction. Subsequently, the analysis pivots to architecting the optimal enterprise-grade, centralized MCP gateway deployment, evaluating modern open-source multiplexers, zero-trust network tunnels, and Docker Compose orchestration to secure and scale remote MCP infrastructure.

## **Baseline Development Environments: Local Client Configurations**

Establishing a functional local development environment requires a granular understanding of how individual AI interfaces process, discover, and execute MCP configurations. Each primary client—Gemini CLI, OpenCode CLI, and GitHub Copilot—utilizes distinct configuration schemas, hierarchical precedence models, and transport protocol implementations.<sup>5</sup>

### **Gemini CLI Configuration Architecture**

Within the Gemini CLI ecosystem, MCP server definitions and global behavioral parameters are governed by the settings.json file, typically located within the .gemini directory in the project root.<sup>8</sup> This configuration is structured primarily through the mcpServers object for explicit server definitions and the mcp object for global governance.<sup>5</sup>

The configuration schema mandates that one of three mutually exclusive transport mechanisms must be defined for each server entry. The command property is utilized for Stdio transport, directing the CLI to spawn a local executable process.<sup>5</sup> Alternatively, the url property defines an endpoint for legacy Server-Sent Events (SSE), while the httpUrl property designates a connection to a modern Streamable HTTP endpoint.<sup>5</sup> When the Gemini CLI initializes, its discovery layer iterates iteratively through the mcpServers object, establishes the designated transport connections, fetches the available tool definitions from the target servers, and registers them dynamically in a global tool registry.<sup>5</sup>

Advanced configurations in the Gemini CLI allow for significant customization. The env object facilitates the passage of environment variables to the spawned server process, supporting cross-platform interpolation syntaxes such as \$VAR\_NAME or \${VAR\_NAME}.<sup>5</sup> The cwd string dictates the working directory context for Stdio transports, while the timeout property dictates the maximum duration for a request, defaulting to a highly permissive 600,000 milliseconds (10 minutes) to accommodate latency-intensive tasks like deep web crawling or complex database queries.<sup>5</sup>

Security and operational automation within Gemini are managed via the trust boolean parameter. If set to true, the CLI bypasses all user confirmation prompts for tool execution originating from that specific server, enabling fully autonomous agentic loops.<sup>5</sup> Furthermore, Gemini supports granular tool filtering through includeTools and excludeTools string arrays, permitting administrators to enforce strict allowlists or blocklists on specific capabilities exposed by an MCP server.<sup>5</sup> Global settings under the top-level mcp object provide an additional layer of strict governance. String arrays such as mcp.allowed and mcp.excluded dictate precisely which server names the CLI is permitted to connect to, overriding any

individual server definitions that might exist in the configuration file.<sup>5</sup>

For sophisticated enterprise environments, particularly those utilizing Google Cloud infrastructure, the Gemini CLI supports complex OAuth and Identity-Aware Proxy (IAP) protection. Configurations can specify an `authProviderType` of `"service_account_impersonation"`, alongside a `targetAudience` and `targetServiceAccount`, to securely route MCP traffic through authenticated cloud boundaries without exposing static credentials.<sup>5</sup> OAuth authentication states can be manually managed via terminal commands such as `gemini mcp auth <serverName>` to initiate browser-based token acquisition or refresh expired sessions.<sup>5</sup>

## OpenCode CLI Configuration Architecture

The OpenCode CLI relies on a distinct configuration file, `opencode.json`, for its MCP routing, characterized by a deep integration with remote organizational endpoints and hierarchical configuration precedence.<sup>7</sup> A defining structural trait of the OpenCode architecture is its capacity to consume default MCP server definitions provided by an organization via a remote `.well-known/opencode` endpoint.<sup>7</sup> These remote defaults are often disabled by default, requiring developers to explicitly opt-in by overriding the remote settings in their local `opencode.json` file with `"enabled": true`.<sup>7</sup>

For authentication and credential management, OpenCode CLI automates OAuth 2.0 flows with high efficiency. If a remote server requires authentication, the CLI automatically detects the requirement and triggers an interactive prompt during the initial tool invocation.<sup>7</sup>

Alternatively, developers can manually initiate the flow using the command `opencode mcp auth <server-name>`.<sup>7</sup> This operation opens a secure browser session for authorization, subsequently storing the resulting access and refresh tokens securely in the user's `~/.local/share/opencode/mcp-auth.json` directory.<sup>7</sup> For servers utilizing static client credentials rather than interactive OAuth, OpenCode allows pre-registered credentials to be hardcoded into the local configuration under an `oauth` object containing `clientId`, `clientSecret` (which can dynamically reference environment variables via `{env:MY_MCP_CLIENT_SECRET}` syntax), and the required OAuth scope strings.<sup>7</sup>

The OpenCode ecosystem is further enhanced by advanced, actively maintained community forks, most notably `Better-OpenCodeMCP`.<sup>9</sup> This fork introduces asynchronous task execution for parallel processing and long-running operations, which is critical for agents performing multiple tool calls simultaneously.<sup>9</sup> It features an interactive setup wizard executed via `node dist/index.js --setup` that fetches available AI models from the local OpenCode installation, allows the user to designate primary and fallback models, and saves these preferences to `~/.config/opencode-mcp/config.json`.<sup>9</sup> The server resolves the active model through a strict hierarchical order: explicit CLI flags (`--model`), the configuration file, auto-detection of the most recently used model, and finally an interactive terminal wizard.<sup>9</sup>

## GitHub Copilot CLI Configuration Architecture

By mid-2026, GitHub Copilot has deeply integrated MCP capabilities into its ecosystem,

featuring a native, built-in official GitHub MCP server that requires zero manual configuration to interact with repositories, pull requests, and issues.<sup>6</sup> However, extending Copilot to utilize external tools requires interacting with the Copilot CLI configuration structure.

Copilot utilizes a manual configuration file located at `~/.copilot/mcp-config.json` or an interactive wizard triggered via the `/mcp add` command in interactive mode.<sup>6</sup> The interactive wizard prompts the user for a unique server name, the server type, and the specific tools to expose (defaulting to `*` for all tools).<sup>6</sup>

The underlying configuration schema supports type declarations of local (STDIO) or http / sse (remote).<sup>6</sup> For local connections, the command and args properties define the executable (e.g., `npx`), while an env object block passes localized environment variables.<sup>6</sup> A notable feature of Copilot's STDIO implementation is that the system PATH variable is inherited automatically, reducing configuration overhead for tools reliant on system binaries.<sup>6</sup>

For http and sse connections, Copilot requires a url property pointing to the remote server and supports a headers object to inject custom HTTP headers, enabling standard API key or Bearer token authentication methodologies (e.g., `{"CONTEXT7_API_KEY": "YOUR-API-KEY"}`).<sup>6</sup> Copilot implements a robust enterprise-level governance layer: if an organization enforces an MCP registry URL and a strict allowlist policy at the enterprise account level, these settings cascade down to the Copilot CLI.<sup>6</sup> The registry URL appears as a mandatory discovery source, and only permitted servers will execute, strictly overriding any local developer configurations.<sup>6</sup>

Furthermore, repository administrators can configure JSON-formatted MCP server details directly within the GitHub.com repository settings under the "Copilot" and "Cloud agent" tabs, allowing the Copilot cloud agent to utilize tools autonomously without local configuration.<sup>11</sup>

## **Local Implementation Phase: Deploying Firecrawl Across Disparate Clients**

To fully grasp the operational friction inherent in decentralized client architectures, an empirical analysis of deploying a specific tool—the Firecrawl MCP server—across all three environments provides critical context. Firecrawl is a specialized, high-performance MCP server that grants AI agents real-time access to the web, allowing them to render complex JavaScript, bypass anti-bot friction, manage retries, and return clean, LLM-ready markdown or structured JSON without manual proxy configurations.<sup>12</sup>

The Firecrawl MCP exposes five primary tools to the AI assistant: Scrape (extracting content from a single URL), Batch Scrape (parallel extraction across multiple URLs), Map (discovering all indexed URLs on a domain), Crawl (systematically walking a site section to extract content), and Search (executing web queries and extracting results).<sup>14</sup> When configured, an AI agent utilizes these tools intelligently; a prompt to "find all blog posts on example.com" will prompt the agent to autonomously call the Map tool to discover the URLs, followed sequentially by the Batch Scrape tool to retrieve the content.<sup>14</sup>

Installing the official server locally is straightforward, requiring Node.js v22+ and executed via the command `npx -y firecrawl-mcp`.<sup>12</sup> A free API key must be generated from the Firecrawl

dashboard.<sup>12</sup> However, without a centralized gateway, a developer must implement the configuration three distinct times, injecting their FIRECRAWL\_API\_KEY into each client's specific schema.

| Client Environment | Configuration File Path    | Implementation Schema Required                                                                                                                              |
|--------------------|----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Gemini CLI         | .gemini/settings.json      | Requires defining an mcpServers object with command: "npx", args: ["-y", "firecrawl-mcp"], and an env dictionary containing FIRECRAWL_API_KEY. <sup>8</sup> |
| OpenCode CLI       | opencode.json              | Requires adding the local tool execution parameters within the mcp registry block, mapping the environment variable appropriately. <sup>7</sup>             |
| GitHub Copilot     | ~/.copilot/mcp-config.json | Requires setting type: "local", command: "npx", args: ["-y", "firecrawl-mcp"], specifying tools: ["*"], and including the env object block. <sup>6</sup>    |

This localized approach creates immediate and severe vulnerabilities. First, the secret FIRECRAWL\_API\_KEY is scattered across multiple plaintext JSON files on the developer's workstation, complicating credential rotation and violating least-privilege principles.<sup>3</sup> Second, updates to the Firecrawl toolset or modifications to caching logic (the server automatically caches responses for 15 minutes to preserve rate limits) require manual synchronization across all three client configurations.<sup>14</sup>

Furthermore, the operational token cost is entirely unmanaged. Because each client operates in isolation, an agent running in Gemini CLI requesting the documentation for a specific framework will consume Firecrawl API credits and LLM tokens to process the scraped markdown. If the developer subsequently asks Copilot the same question, the entire expensive process is repeated because there is no shared semantic cache or centralized state.<sup>3</sup> On Windows environments, local execution introduces additional friction, often requiring command workarounds such as `cmd /c "set FIRECRAWL_API_KEY=your-api-key && npx -y firecrawl-mcp"` to execute properly.<sup>15</sup>

The analytical conclusion derived from this deployment exercise is absolute: while localized configurations serve effectively for rapid prototyping, they are structurally and economically unsuitable for continuous enterprise production.

## The Architectural Shift: Centralized MCP Gateways

To resolve the fragmentation, security risks, and economic inefficiencies of localized configurations, the enterprise standard has shifted entirely to deploying MCP Gateways and Multiplexers. An MCP gateway functions as a highly sophisticated, dual-mode reverse proxy situated between the AI clients and the backend tool infrastructure.<sup>1</sup>

In its capacity as an MCP client, the gateway connects securely to various backend tool servers—such as Firecrawl for web scraping, internal PostgreSQL databases, Jira for issue tracking, and custom business logic APIs—utilizing optimized transport protocols like Stdio, HTTP, or SSE.<sup>1</sup> Simultaneously, in its capacity as an MCP server, the gateway exposes this aggregated catalog of tools to downstream AI interfaces (Copilot, Gemini, OpenCode) through a single, unified, and highly governed endpoint.<sup>1</sup>

This architecture yields profound second-order and third-order benefits. It consolidates credential management into a single secure vault, entirely preventing the exposure of raw backend API keys or database connection strings to the external LLM or the end-user's local machine.<sup>1</sup> It provides a singular locus for Role-Based Access Control (RBAC), allowing administrators to dictate precisely which tools specific users or teams can execute.<sup>3</sup>

Furthermore, it provides holistic observability into tool execution, capturing comprehensive audit logs of every prompt and subsequent tool call.<sup>1</sup>

Crucially, centralized gateways solve the "token explosion" problem inherent in agentic AI. MCP tool responses, particularly from tools like Firecrawl or robust database queries, can be excessively verbose, consuming massive amounts of the LLM's context window. Advanced commercial and open-source gateways parse and compress these outputs, reducing token consumption by 40% to 60% before returning the payload to the LLM, thereby drastically reducing inference costs.<sup>3</sup>

## **Comprehensive Evaluation of Open-Source MCP Gateways (May 2026)**

The ecosystem for open-source MCP gateways has diversified and matured rapidly. Selecting the optimal software for a self-hosted Docker Compose deployment requires evaluating the distinct architectural philosophies, performance benchmarks, and feature sets of the leading platforms.

### **1. Bifrost Gateway**

Bifrost has firmly established itself as the premier high-performance AI gateway for enterprise production. Written in Go, its defining architectural trait is the complete unification of LLM routing and MCP tool access through a single, OpenAI-compatible API across more than 12 model providers.<sup>16</sup>

Performance benchmarks for Bifrost are exceptional. It operates with a minimal latency overhead of merely 11 microseconds (~11µs), maintaining a 100% request success rate even under sustained loads exceeding 5,000 requests per second (RPS).<sup>16</sup> In comparative benchmarking against legacy Python-based gateways, Bifrost delivers 9.5x more throughput and 54x lower P99 latency.<sup>17</sup>

Beyond raw cryptographic and routing speed, Bifrost introduces sophisticated, agent-specific capabilities. Its native MCP gateway acts simultaneously as a client and server, centralizing authentication, authorization, and tool discovery into a single control layer.<sup>19</sup> Crucially, Bifrost implements true Semantic Caching.<sup>16</sup> Unlike basic exact-match caching, Semantic Caching utilizes embedding models to identify semantically identical tool queries—even if the LLM formulates the request with slight syntactic variations—and serves the cached tool response.<sup>16</sup> This intercepts redundant external API calls to tools like Firecrawl and drastically lowers token spend for applications exhibiting repetitive query patterns.<sup>21</sup> Bifrost further enhances governance by featuring hierarchical budget management (capping spend at the team or project level), native Prometheus OpenTelemetry metrics, and HashiCorp Vault integration for centralized cryptographic key management.<sup>16</sup>

## 2. LiteLLM Proxy (Agent & MCP Gateway)

LiteLLM operates as a highly adopted, Python-based open-source gateway. Originally designed to unify access to over 100 LLM providers via a consistent OpenAI format, it has since natively integrated a comprehensive MCP Gateway into its proxy architecture.<sup>22</sup>

LiteLLM is configured via a robust `config.yaml` file, supporting a unified control plane where LLMs, MCP servers, and autonomous agent routes share identical authentication, rate limiting, and observability parameters.<sup>23</sup> A standout, highly differentiated feature of LiteLLM is its ability to automatically convert any existing OpenAPI 3.x or Swagger 2.0 specification directly into an MCP server without requiring the deployment of custom server code.<sup>24</sup> It intelligently translates the `operationId` parameters from the specification into executable tool names, providing seamless integration for legacy REST APIs.<sup>24</sup> Developers can further customize the agent experience by overriding default tool names and descriptions via the LiteLLM UI or API, utilizing the `tool_name_to_display_name` parameter.<sup>24</sup>

LiteLLM provides profound granularity in access control and security. Administrators can map `mcp_aliases` to resolve namespace collisions between servers offering similarly named tools.<sup>24</sup>

They can implement strict `allowed_tools` and `disallowed_tools` string arrays to whitelist or blacklist specific functions, and even restrict the specific parameters that an agent is allowed to pass to a tool via the `allowed_params` dictionary.<sup>24</sup>

For authentication against backend servers, LiteLLM supports extensive protocols including API keys, Bearer tokens, Basic Auth, OAuth2, and AWS SigV4 for dynamic request signing against AWS Bedrock AgentCore.<sup>24</sup>

Furthermore, LiteLLM excels in troubleshooting capabilities. By injecting a specific curl header (`x-litellm-mcp-debug: true`), administrators can retrieve masked diagnostic headers such as `x-mcp-debug-oauth2-token` and `x-mcp-debug-inbound-auth`.<sup>24</sup> These diagnostics allow rapid identification of token leakage—for example, if the value `SAME_AS_LITELLM_KEY` appears in the OAuth header, it indicates that the master API key is inadvertently leaking to downstream MCP servers instead of a properly scoped OAuth2 token, enabling immediate remediation.<sup>24</sup>

However, its Python-based architecture incurs moderate latency overheads compared to Go-based alternatives, and its caching mechanism relies on exact-match rather than semantic similarity.<sup>21</sup>

### 3. Docker MCP Gateway

The official Docker MCP Gateway (`docker/mcp-gateway`) is an open-source, container-native proxy explicitly designed for engineering teams heavily invested in the Docker ecosystem.<sup>3</sup> It utilizes a centralized `catalog.yaml` file to define and register all upstream server connections.<sup>3</sup> The paramount strength of the Docker gateway is its strict container isolation strategy.<sup>27</sup> By securely mounting the Docker socket (`/var/run/docker.sock`), the gateway dynamically spins up and manages containerized MCP servers on demand. This ensures that the execution of arbitrary, potentially dangerous tools—such as direct filesystem interactions or shell command executions initiated by the LLM—remains entirely sandboxed from the host operating system, mitigating severe security risks.<sup>3</sup>

However, the configuration schema is notoriously strict and unforgiving. Every server entry defined in the `catalog.yaml` must explicitly declare the boolean `enabled: true`; failing this, the gateway defaults to a disabled state and silently ignores the server, a frequent source of deployment failures.<sup>3</sup> The architecture natively supports `stdio`, `sse`, and `http` transports.<sup>3</sup> While highly secure and robust for local development, the Docker MCP Gateway introduces a latency overhead of 50 to 200 milliseconds, primarily due to the computational overhead of Docker socket translation, IPC communication, and container spawning.<sup>26</sup> Furthermore, it acts strictly as a protocol translator for tools and lacks the native LLM routing and semantic caching capabilities found in Bifrost or LiteLLM.<sup>3</sup>

### 4. Specialized Niche Gateways

Beyond the primary multiplexers, several open-source gateways serve specific niche architectural requirements:

- **ContextForge (IBM):** Engineered for platform teams requiring open-source multi-protocol federation. It excels at translating REST and gRPC payloads directly into MCP tool signatures. However, it relies heavily on Kubernetes and over 40 plugin extensions, making it powerful but operationally complex, and it lacks built-in semantic caching.<sup>16</sup>
- **MCPJungle:** A lightweight gateway focused heavily on acting as an enterprise registry. It bundles tool discovery with standard RBAC, deployable via a single binary or Docker Compose, making it ideal for smaller teams needing immediate governance without complex LLM routing.<sup>20</sup>
- **Traefik Hub MCP Gateway:** Utilizes a "Triple Gate Pattern" to layer MCP capabilities directly over existing Traefik ingress infrastructure, simultaneously protecting the model, protocol, and API layers using Task-Based Access Control (TBAC).<sup>16</sup>
- **MCPO (OpenWebUI MCP Proxy):** Functions as an inverse proxy, converting MCP servers into HTTP/OpenAPI endpoints, enabling usage from traditional UIs that lack native MCP integration.<sup>20</sup>
- **DeployStack:** A team-focused gateway providing a credential vault and RBAC, notable for its 2-tool hierarchical router that claims up to 98% token reduction by aggressively



managing tool payloads.<sup>20</sup>

| Gateway Platform   | Primary Architecture | p95 Latency Overhead | Semantic Caching | Unified LLM + MCP | Core Differentiator (2026)                                                     |
|--------------------|----------------------|----------------------|------------------|-------------------|--------------------------------------------------------------------------------|
| Bifrost            | Go                   | ~11µs                | Supported        | Yes               | Ultra-low latency, native LLM routing, and hierarchical budgets. <sup>16</sup> |
| LiteLLM Proxy      | Python               | Moderate             | No (Exact Match) | Yes               | Instant OpenAPI-to-MCP translation, granular parameter control. <sup>21</sup>  |
| Docker MCP Gateway | Go / Node            | 50-200ms             | No               | No                | Strict container isolation and dynamic Docker socket management. <sup>3</sup>  |
| ContextForge       | Multi                | Moderate             | No               | No                | Multi-protocol translation (REST/gRPC) across Kubernetes. <sup>16</sup>        |
| MCPJungle          | Unknown              | Low                  | No               | No                | Lightweight built-in registry with enterprise RBAC mode. <sup>29</sup>         |

# Architecting the Self-Hosted Remote Docker Compose Deployment

Based on the empirical evidence from 2026 enterprise architectures, the optimal self-hosted open-source solution to host remote MCP servers utilizes **Bifrost** as the core LLM and MCP multiplexer, isolated within a secure, multi-layered Docker Compose environment. This ensures ultra-low latency execution, semantic cache optimizations, unified observability, and strict network isolation.<sup>16</sup>

## Component Architecture and Network Topology

The deployment architecture requires three logically separated layers, orchestrated via `docker-compose.yml`:

1. **The Identity and Access Layer:** Components managing authentication, Single Sign-On (SSO), and network tunneling to prevent public exposure.
2. **The Gateway Control Plane (Bifrost):** The central node performing dual-mode MCP translation, API routing, token compression, and semantic caching.
3. **The Backend Tool Layer:** Highly isolated Docker containers hosting the actual MCP server instances (e.g., Firecrawl container, GitHub server, internal PostgreSQL access tools).

The architectural logic of the `docker-compose.yml` must strictly enforce network isolation bridging.<sup>30</sup> Backend MCP servers must be attached to an internal Docker bridge network (e.g., `mcp-internal`), entirely devoid of external host port bindings. For example, a localized Firecrawl container (`ghcr.io/firecrawl/firecrawl-mcp`) and a local database tool container would exist exclusively on this internal network.

The Bifrost gateway container acts as the bridge, attached to both the `mcp-internal` network and a restricted gateway-external network. Bifrost is configured to connect to the backend tools via Streamable HTTP or SSE over the internal bridge, eliminating the complexities and security risks of Stdio transport across container boundaries, which often fails in cloud environments.<sup>3</sup>

The gateway utilizes a declarative configuration file or a local database volume to map the internal tool endpoints. It centralizes all required API credentials—for instance, injecting the `FIRECRAWL_API_KEY` into the Firecrawl container via Docker secrets or an integrated HashiCorp Vault, rather than managing them through client-side JSON files.<sup>16</sup>

By unifying the backend tools behind Bifrost, the end-user clients (Gemini CLI, OpenCode, Copilot) are pointed exclusively to the Bifrost SSE endpoint (`http://bifrost:8080/mcp`). When the AI client requests a tool execution, Bifrost authenticates the client request, routes the LLM intent, queries the semantic cache for previous identical operations, calls the relevant isolated backend container over the internal network, compresses the tool output, and streams the optimized result back to the client.<sup>1</sup>

If adopting the Docker MCP Gateway instead of Bifrost for pure tool isolation, the `docker-compose.yml` must mount the catalog file as read-only (`-v`

`/app/catalogs.yaml:/app/catalogs.yaml:ro`) and securely mount the Docker socket.<sup>31</sup> In

production, mounting the raw Docker socket grants root-equivalent access, creating a severe vulnerability. Security best practices dictate running the gateway with a non-root user (user: `"1000:1000"`), passing the `--userns-remap` flag, and utilizing a Docker socket proxy (such as Traefik or a dedicated HAProxy instance) to enforce read-only access to the socket.<sup>3</sup>

Furthermore, health checks utilizing the `/health` endpoint must be integrated into the compose file to ensure automatic container recovery.<sup>3</sup>

## Enterprise Security Posture and Zero-Trust Network

# Tunneling

Exposing an MCP gateway to external AI clients or developer workstations introduces critical security vectors. Because MCP servers operate with delegated user permissions and execute chained tool calls autonomously, a compromised or spoofed gateway acts as an unauthorized, highly privileged control plane for enterprise infrastructure.<sup>32</sup> The Open Worldwide Application Security Project (OWASP) dictates that secure MCP architectures require rigorous endpoint verification, strictly scoped permissions, and continuous behavioral auditing of the agent-server dialogue.<sup>32</sup>

## Authentication: OIDC and Token Exchange

Relying purely on static API keys for gateway access is a deprecated, high-risk practice. Enterprise architectures in 2026 mandate OpenID Connect (OIDC) and OAuth 2.1 integration for MCP multiplexers.<sup>34</sup>

Integrating an open-source identity provider like Keycloak into the Docker Compose stack enables Dynamic Client Registration and the enforcement of fine-grained token scopes.<sup>34</sup> In this topology, AI agents are issued short-lived OAuth2 access tokens. However, a significant vulnerability arises if these broad-scoped agent tokens are passed directly to backend MCP servers; a compromised backend server could harvest the token to impersonate the agent and access unauthorized resources.<sup>36</sup>

To mitigate this, advanced deployments utilize a proxy add-on, such as Kuadrant or the native auth modules in Bifrost and LiteLLM, to perform cryptographic "Token Exchange".<sup>36</sup> The gateway validates the inbound agent token against Keycloak, verifies the agent's specific RBAC permissions to execute the requested tools, and then initiates the backend tool calls using separate, narrowly scoped credentials specific only to that target server.<sup>24</sup> This ensures complete session isolation.

## Network Tunnels: Tailscale vs. Cloudflare Tunnels

To securely connect distributed developer machines to the remote Docker Compose gateway without exposing it to the public internet, organizations must deploy secure last-mile network tunnels. The primary architectural debate centers on Tailscale versus Cloudflare Tunnels.<sup>37</sup>

Cloudflare Tunnels function as an outbound reverse proxy, creating a secure, encrypted connection between the Docker Compose infrastructure and Cloudflare's global edge network without requiring the opening of inbound firewall ports.<sup>39</sup> Users authenticate via Cloudflare Access (Zero Trust) before traffic is permitted to reach the gateway.<sup>38</sup> While highly effective for exposing specific, low-risk webhooks to third-party integrations, it mandates that all internal enterprise MCP traffic routes through Cloudflare's servers. This introduces latency overheads and potential data-residency compliance issues under stringent regulatory frameworks.<sup>38</sup>

Tailscale, conversely, utilizes a fundamentally different architecture based on the WireGuard VPN protocol.<sup>2</sup> Tailscale constructs an encrypted, peer-to-peer mesh network directly between authenticated devices. Traffic flowing between a developer's laptop running Copilot

and the remote Bifrost gateway travels directly over the shortest possible network path, rather than being relayed through a centralized corporate hub.<sup>2</sup>

For internal enterprise AI deployments and sensitive MCP traffic, Tailscale is structurally superior.<sup>2</sup> It eliminates public exposure entirely; the MCP gateway only binds to the internal Tailscale virtual network interface (typically 100.x.x.x). A unified Zero Trust agent, enforced via Mobile Device Management (MDM) platforms, handles local machine health checks before allowing the connection.<sup>37</sup> This absolute zero-exposure posture ensures that automated scanners, malicious bots, and unauthenticated public endpoints cannot even ping the gateway, vastly reducing the risk of unauthorized tool discovery or execution.<sup>2</sup>

## Behavioral Intent Detection and OWASP Guidelines

Once the network perimeter is secured via Tailscale and identity is verified via Keycloak, security must be aggressively enforced at the runtime execution level. Gateways must implement logic to protect against malicious payloads, prompt injection, and agent hallucinations.

1. **Semantic Rate Limiting:** Traditional volumetric rate limiting (e.g., 100 requests per minute) is insufficient for autonomous AI. If an agent enters a hallucination loop and attempts to execute a `read_file` tool 500 times in ten seconds, the gateway must dynamically terminate the connection, recognizing the behavior as an abnormal execution pattern rather than legitimate intent.<sup>2</sup>
2. **Human-In-The-Loop (HITL) Triggers:** The MCP specification explicitly recommends, and OWASP guidelines dictate, that any state-altering operation (e.g., `write_file`, `execute_command`, `create_records_for_table`, or database DROP statements) must automatically halt the agent workflow.<sup>2</sup> The gateway must require manual cryptographic approval from the human operator before allowing the tool payload to proceed to the backend server.<sup>2</sup>
3. **Prompt Injection Sanitization:** All MCP tool descriptions, system prompts, and payload arguments must be treated inherently as untrusted input. The gateway must sanitize and validate arguments before passing them into the model context or executing the backend function.<sup>37</sup> Tools like Lasso Security or TrueFoundry MCP Gateway are frequently layered into the stack to provide active prompt injection detection and automatic, traceable tool-call interaction logging.<sup>40</sup>

## Second and Third-Order Implications of MCP Centralization

The migration from localized, decentralized JSON configurations to centralized, secure MCP multiplexers triggers cascading structural effects across enterprise architecture.

### The Decoupling of LLMs and Tool Execution

Historically, AI capabilities were intimately tied to specific model providers; building an agent required deep integration with proprietary APIs, such as OpenAI's function calling architecture.

By standardizing entirely on the Model Context Protocol and centralizing it behind an intelligent gateway like Bifrost or LiteLLM, organizations achieve total model agnosticism.<sup>1</sup> Because the gateway dynamically manages tool descriptions, parses arguments, and translates responses into a unified OpenAI-compatible format, the coupling between the brain (the LLM) and the hands (the tools) is broken.<sup>41</sup> Developers can seamlessly route complex data-extraction tasks to heavy frontier models (like GPT-4o) and simpler, high-volume tasks to faster, cheaper models (like Claude 3 Haiku or self-hosted Llama 3 instances) without altering a single line of client configuration.<sup>25</sup> This breaks vendor lock-in and commoditizes the LLM routing layer entirely.

## **The Shift in AI Governance**

Centralization fundamentally alters the paradigm of AI governance. When developers inject their personal API keys into local settings.json files, security and compliance teams have absolute zero visibility into what proprietary data the LLMs are ingesting or what actions they are executing.

A unified gateway shifts governance from a reactive posture (auditing exposed API keys post-breach) to a proactive, real-time posture (monitoring agent intent as it happens).<sup>2</sup> Security teams can enforce strict Data Loss Prevention (DLP) policies directly at the gateway layer. They can completely disable specific tools using blocklists, or enforce strict parameter constraints across an entire organizational unit, ensuring an agent can read a database but never write to it.<sup>24</sup> The gateway becomes the absolute source of truth for agent activity, capturing every span, session, and tool trace for OpenTelemetry compatibility, enabling comprehensive post-incident forensics.<sup>18</sup>

## **Operational Economics and Token Optimization**

The token overhead associated with extensive tool descriptions, system prompts, and highly verbose API responses is a primary driver of escalating enterprise AI costs. Centralized gateways mitigate this severe economic strain through dual mechanisms. Output compression algorithms parse raw data—such as a massive, unstructured HTML dump returned by the Firecrawl Scrape tool—and reduce the token footprint significantly by stripping out unnecessary boilerplate before returning the payload to the LLM.<sup>3</sup>

Furthermore, semantic caching drastically reduces redundant API calls.<sup>16</sup> If five different developers ask Copilot to "summarize the latest GitHub deployment logs," a gateway like Bifrost recognizes the semantic intent underlying the varying prompts. It serves the cached tool response instantly, eliminating five identical LLM generation cycles and five identical downstream API requests to GitHub.<sup>16</sup> At scale, this reduces API rate-limit throttling and cuts inference costs dramatically.

## **Conclusion**

The landscape of agentic AI infrastructure in 2026 demands the immediate deprecation of localized, client-side MCP configurations. Attempting to manage disjointed configurations

across Gemini CLI, OpenCode CLI, and GitHub Copilot results in unmanageable credential sprawl, fragmented toolsets, unoptimized token expenditure, and severe security vulnerabilities.<sup>3</sup>

To establish a highly available, secure, and economically performant remote development environment, enterprises must deploy an open-source MCP Gateway within a hardened Docker Compose ecosystem.

The empirical evidence strongly recommends **Bifrost** as the primary multiplexer due to its 11μs latency, advanced semantic caching, and unified dual-mode architecture that seamlessly orchestrates both LLM routing and MCP tool execution.<sup>16</sup> For organizations heavily invested in translating legacy OpenAPI specifications into modern MCP tools, **LiteLLM** serves as a highly capable, granular alternative.<sup>24</sup>

Crucially, this gateway infrastructure must be sequestered behind a **Tailscale** peer-to-peer mesh network, entirely eliminating public IP exposure and ensuring that only MDM-compliant, authenticated developer machines can reach the unified MCP endpoint.<sup>2</sup> Authentication and Role-Based Access Control should be delegated to an OIDC provider like **Keycloak** to enforce dynamic client registration and prevent lateral token leakage via token exchange protocols.<sup>34</sup> Finally, the gateway must strictly adhere to OWASP security guidelines, applying semantic rate limiting, proactive prompt injection sanitization, and mandatory Human-in-the-Loop confirmations for all state-altering operations.<sup>2</sup>

By adopting this centralized, secure architectural framework, organizations resolve the configuration drift dilemma and transform disconnected AI coding assistants into a highly governed, economically optimized, and deeply integrated enterprise intelligence layer.

## Works cited

1. Best MCP Gateway in 2026 for Enterprise AI Applications : r/LLM\_Gateways - Reddit, accessed on May 16, 2026, [https://www.reddit.com/r/LLM\\_Gateways/comments/1s3hhol/best\\_mcp\\_gateway\\_in\\_2026\\_for\\_enterprise\\_ai/](https://www.reddit.com/r/LLM_Gateways/comments/1s3hhol/best_mcp_gateway_in_2026_for_enterprise_ai/)
2. Securing MCP Servers: The 2026 Guide to AI Tool Tunneling | by InstaTunnel - Medium, accessed on May 16, 2026, <https://medium.com/@instatunnel/securing-mcp-servers-the-2026-guide-to-ai-tool-tunneling-aafa113b08db>
3. Deploy an MCP Gateway with Docker in 5 Minutes (2026) | Apigene Blog, accessed on May 16, 2026, <https://apigene.ai/blog/deploy-mcp-gateway-docker-5-minutes>
4. Best MCP Gateways for Self-Hosted Deployments 2026 | MintMCP Blog, accessed on May 16, 2026, <https://www.mintmcp.com/blog/mcp-gateways-self-hosted-deployments>
5. MCP servers with Gemini CLI | Gemini CLI, accessed on May 16, 2026, <https://geminicli.com/docs/tools/mcp-server/>
6. Adding MCP servers for GitHub Copilot CLI - GitHub Docs, accessed on May 16, 2026,

- <https://docs.github.com/en/copilot/how-tos/copilot-cli/customize-copilot/add-mcp-servers>
7. MCP servers | OpenCode, accessed on May 16, 2026,  
<https://opencode.ai/docs/mcp-servers/>
  8. How to Build an MCP Server with Gemini CLI and Go | Google Codelabs, accessed on May 16, 2026,  
<https://codelabs.developers.google.com/cloud-gemini-cli-mcp-go>
  9. GitHub - ajhcs/Better-OpenCodeMCP: MCP server for OpenCode CLI — async task execution, process pooling, crash recovery, and 293 tests. Bridge Claude to Gemini, DeepSeek, and other models., accessed on May 16, 2026,  
<https://github.com/ajhcs/Better-OpenCodeMCP>
  10. Setting up the GitHub MCP Server, accessed on May 16, 2026,  
<https://docs.github.com/en/copilot/how-tos/provide-context/use-mcp-in-your-ide/set-up-the-github-mcp-server>
  11. Connect agents to external tools - GitHub Docs, accessed on May 16, 2026,  
<https://docs.github.com/en/copilot/how-tos/copilot-on-github/customize-copilot/customize-cloud-agent/extend-cloud-agent-with-mcp>
  12. How to Set Up and Use Firecrawl MCP in Cursor, accessed on May 16, 2026,  
<https://www.firecrawl.dev/blog/firecrawl-mcp-in-cursor>
  13. Live Web Data for AI Agents | Firecrawl MCP for Claude Code & Cursor, accessed on May 16, 2026, <https://www.firecrawl.dev/use-cases/ai-mcps>
  14. Developers & MCP - Firecrawl Docs, accessed on May 16, 2026,  
<https://docs.firecrawl.dev/use-cases/developers-mcp>
  15. Official Firecrawl MCP Server - Adds powerful web scraping and search to Cursor, Claude and any other LLM clients. - GitHub, accessed on May 16, 2026,  
<https://github.com/firecrawl/firecrawl-mcp-server>
  16. Top 5 MCP Gateways in 2026 - Maxim AI, accessed on May 16, 2026,  
<https://www.getmaxim.ai/articles/top-5-mcp-gateways-in-2026-3/>
  17. Fastest LLM Gateway: Bifrost vs LiteLLM - Artificial Intelligence in Plain English, accessed on May 16, 2026,  
<https://ai.plainenglish.io/i-switched-my-llm-gateway-and-got-50x-faster-heres-what-nobody-tells-you-b2346f069f01>
  18. Top 5 LLM Gateways for 2026: A Comprehensive Comparison - Maxim AI, accessed on May 16, 2026,  
<https://www.getmaxim.ai/articles/top-5-llm-gateways-for-2026-a-comprehensive-comparison/>
  19. Bifrost Alternatives: Top Tools You Can Consider in 2026 - Truefoundry, accessed on May 16, 2026,  
<https://www.truefoundry.com/blog/bifrost-alternative-mcp-gateway>
  20. What is the best MCP GATEWAY? - Reddit, accessed on May 16, 2026,  
[https://www.reddit.com/r/mcp/comments/1q8fmmg/what\\_is\\_the\\_best\\_mcp\\_gateway/](https://www.reddit.com/r/mcp/comments/1q8fmmg/what_is_the_best_mcp_gateway/)
  21. LiteLLM vs Bifrost: Which AI Gateway Is Right for Enterprise Teams? - DEV Community, accessed on May 16, 2026,  
[https://dev.to/kuldeep\\_paul/litellm-vs-bifrost-which-ai-gateway-is-right-for-enter](https://dev.to/kuldeep_paul/litellm-vs-bifrost-which-ai-gateway-is-right-for-enter)

- [prise-teams-4h2f](#)
22. Getting Started - LiteLLM Docs, accessed on May 16, 2026, <https://docs.litellm.ai/docs/>
  23. MCP Deployment Guide - liteLLM, accessed on May 16, 2026, [https://docs.litellm.ai/docs/mcp\\_deployment](https://docs.litellm.ai/docs/mcp_deployment)
  24. MCP Overview | liteLLM, accessed on May 16, 2026, <https://docs.litellm.ai/docs/mcp>
  25. Using your MCP - LiteLLM Docs, accessed on May 16, 2026, [https://docs.litellm.ai/docs/mcp\\_usage](https://docs.litellm.ai/docs/mcp_usage)
  26. 10 Best MCP Gateways for Developers in 2026: A Deep Dive Comparison - Composio, accessed on May 16, 2026, <https://composio.dev/content/best-mcp-gateway-for-developers>
  27. How to build and deliver an MCP server for production - Docker, accessed on May 16, 2026, <https://www.docker.com/blog/build-to-prod-mcp-servers-with-docker/>
  28. Best MCP Gateways and AI Agent Security Tools (2026) - Integrate.io, accessed on May 16, 2026, <https://www.integrate.io/blog/best-mcp-gateways-and-ai-agent-security-tools/>
  29. Best Open Source MCP Gateways 2026 - Lunar.dev, accessed on May 16, 2026, <https://www.lunar.dev/post/the-best-open-source-mcp-gateways-in-2026>
  30. Manage multiple MCP servers for Ollama + OpenWebUI as Docker service - Reddit, accessed on May 16, 2026, [https://www.reddit.com/r/LocalLLaMA/comments/1lx0b5w/manage\\_multiple\\_mcp\\_servers\\_for\\_ollama\\_openwebui/](https://www.reddit.com/r/LocalLLaMA/comments/1lx0b5w/manage_multiple_mcp_servers_for_ollama_openwebui/)
  31. Running local private catalog seems to only support a single MCP server? · Issue #126 · docker/mcp-gateway - GitHub, accessed on May 16, 2026, <https://github.com/docker/mcp-gateway/issues/126>
  32. A Practical Guide for Secure MCP Server Development - OWASP Gen AI Security Project, accessed on May 16, 2026, <https://genai.owasp.org/resource/a-practical-guide-for-secure-mcp-server-development/>
  33. MCP Security Issues and Best Practices You Need to Know - Knostic, accessed on May 16, 2026, <https://www.knostic.ai/blog/mcp-security>
  34. oidebrett/mcpauth: MCP Gateway for external Authentication and Authorization - GitHub, accessed on May 16, 2026, <https://github.com/oidebrett/mcpauth>
  35. Understanding Authorization in MCP - Model Context Protocol, accessed on May 16, 2026, <https://modelcontextprotocol.io/docs/tutorials/security/authorization>
  36. Advanced authentication and authorization for MCP Gateway | Red Hat Developer, accessed on May 16, 2026, <https://developers.redhat.com/articles/2025/12/12/advanced-authentication-authorization-mcp-gateway>
  37. Securing the Last Mile: Implementing Compliance-Gated Tunnels in 2026 | by InstaTunnel, accessed on May 16, 2026, <https://medium.com/@instatunnel/securing-the-last-mile-implementing-compliance-gated-tunnels-in-2026-add57585cebd>



38. Cloudflare vs. Tailscale | Compare Access and Gateway to Tailscale, accessed on May 16, 2026, <https://tailscale.com/compare/cloudflare-access>
39. hhftechnology/cloudflare-tailscale-integration: Zero-trust access gateway: Deploy secure tunnels for both public and private services instantly. - GitHub, accessed on May 16, 2026, <https://github.com/hhftechnology/cloudflare-tailscale-integration>
40. Best MCP Security Tools in 2026 - Truefoundry, accessed on May 16, 2026, <https://www.truefoundry.com/blog/best-mcp-security-tools>
41. Building a Production-Ready Code Review Agent with Bifrost's MCP Integration - Medium, accessed on May 16, 2026, <https://medium.com/@kuldeep.paul08/building-a-production-ready-code-review-agent-with-bifrosts-mcp-integration-b58a104d790c>